

Hybrid CNN-BiLSTM-Attention - Line by Line

Class Definition

```
1 class HybridCNNBiLSTMAttention(nn.Module):
```

Creates hybrid model combining CNN, BiLSTM, and Attention.

Constructor

```
1     def __init__(self, vocab_size, embedding_dim=256, n_filters=64,  
2                     filter_sizes=[3, 4, 5], hidden_dim=128, num_layers=2,  
3                     output_dim=3, dropout=0.5):
```

Initialization with parameters:

- vocab_size: Number of unique words
- embedding_dim=256: Word vector size
- n_filters=64: Filters per CNN kernel
- filter_sizes=[3, 4, 5]: CNN kernel sizes
- hidden_dim=128: BiLSTM hidden size per direction
- num_layers=2: BiLSTM layers
- output_dim=3: Sentiment classes
- dropout=0.5: Dropout rate

```
1     super().__init__()
```

Initializes parent class.

```
1     self.embedding = nn.Embedding(vocab_size, embedding_dim,  
                                   padding_idx=0)
```

Creates embedding layer.

```
1     self.convs = nn.ModuleList([  
2         nn.Conv1d(  
3             in_channels=embedding_dim,  
4             out_channels=n_filters,  
5             kernel_size=fs,  
6             padding=fs//2  
7         )  
8         for fs in filter_sizes  
9     ])
```

Creates 3 Conv1D layers with padding to preserve sequence length:

- `padding=fs//2`: Adds padding (e.g., 1 for kernel 3, 2 for kernel 5)
- Maintains same sequence length after convolution
- Each conv: 256 input channels, 64 output channels

```
1     self.bilstm = nn.LSTM(  
2         input_size=embedding_dim,  
3         hidden_size=hidden_dim,  
4         num_layers=num_layers,  
5         batch_first=True,  
6         dropout=dropout if num_layers > 1 else 0,  
7         bidirectional=True  
8     )
```

Creates bidirectional LSTM with 2 layers, hidden size 128 per direction (256 total).

```
1     self.attention = nn.Sequential(  
2         nn.Linear(hidden_dim * 2, hidden_dim),  
3         nn.Tanh(),  
4         nn.Linear(hidden_dim, 1)  
5     )
```

Creates attention mechanism: Linear ($256 \rightarrow 128$) $\rightarrow$ Tanh $\rightarrow$ Linear ($128 \rightarrow 1$).

```
1     self.dropout = nn.Dropout(dropout)
```

Creates dropout layer.

```
1     total_features = (n_filters * len(filter_sizes)) + (hidden_dim  
2         * 2)
```

Calculates total features: CNN features ($64 \times 3 = 192$) + LSTM features ($128 \times 2 = 256$) = 448.

```
1     self.fc = nn.Linear(total_features, output_dim)
```

Creates output layer ($448 \rightarrow 3$).

```
1     self._init_weights()
```

Calls weight initialization.

Weight Initialization Method

```
1     def _init_weights(self):  
2         for conv in self.convs:  
3             nn.init.xavier_uniform_(conv.weight)  
4             nn.init.xavier_uniform_(self.fc.weight)
```

Initializes CNN and FC layer weights using Xavier uniform.

Forward Method

```
1 def forward(self, input_ids, attention_mask):
```

Defines forward pass with required inputs.

```
1     embedded = self.embedding(input_ids)
2     embedded = self.dropout(embedded)
```

Converts to embeddings and applies dropout. Shape: (batch, sequence, 256).

```
1     embedded_cnn = embedded.permute(0, 2, 1)
```

Transposes for CNN. Shape: (batch, 256, sequence).

```
1     conved = [F.relu(conv(embedded_cnn)) for conv in self.convs]
```

Applies each convolution with ReLU. Creates 3 tensors, each shape: (batch, 64, sequence).

```
1     pooled = [F.max_pool1d(conv, conv.shape[2]).squeeze(2) for conv
                in conved]
```

Global max pooling on each. Shape per tensor: (batch, 64).

```
1     cnn_features = torch.cat(pooled, dim=1)
2     cnn_features = self.dropout(cnn_features)
```

Concatenates CNN features and applies dropout. Shape: (batch, 192).

```
1     bilstm_out, _ = self.bilstm(embedded)
```

Processes through BiLSTM. Shape: (batch, sequence, 256).

```
1     attn_scores = self.attention(bilstm_out).squeeze(-1)
```

Computes attention scores. Shape: (batch, sequence).

```
1     attn_scores = attn_scores.masked_fill(attention_mask == 0, -1e9
2     )
```

Masks padding positions.

```
1     attn_weights = F.softmax(attn_scores, dim=1).unsqueeze(1)
```

Converts to probabilities. Shape: (batch, 1, sequence).

```
1     lstm_features = torch.bmm(attn_weights, bilstm_out).squeeze(1)
2     lstm_features = self.dropout(lstm_features)
```

Computes weighted context vector and applies dropout. Shape: (batch, 256).

```
1     combined = torch.cat([cnn_features, lstm_features], dim=1)
```

Concatenates CNN and LSTM features. Shape: (batch, 448).

```
1     return self.fc(combined)
```

Produces 3 output logits.

Model Instantiation

```
1 model = HybridCNNBiLSTMAttention(  
2     vocab_size=len(vocab),  
3     embedding_dim=256,  
4     n_filters=64,  
5     filter_sizes=[3, 4, 5],  
6     hidden_dim=128,  
7     num_layers=2,  
8     output_dim=3,  
9     dropout=0.5  
10 ).to(device)
```

Creates model instance and moves to device.

```
1 print(f" Modele Hybride CNN + BiLSTM + Attention initialise :")  
2 print(f"     - Embedding dim      : 256")  
3 print(f"     - CNN Filtres           : 64      3 tailles (3,4,5)")  
4 print(f"     - BiLSTM Hidden dim      : 128 ( 2  bidirectionnel)")  
5 print(f"     - Fusion                  : Parallele (CNN features + LSTM context  
6         )")  
6 print(f"     - Parametres              : {sum(p.numel() for p in model.  
    parameters()):,}")
```

Prints model configuration and parameter count.

Tools Used

PyTorch Core (torch)

- `torch.cat`: Concatenates tensors
- `torch.bmm`: Batch matrix multiplication

Neural Network Module (nn)

- `nn.Module`: Base class
- `nn.Embedding`: Word embeddings
- `nn.ModuleList`: Container for CNN layers
- `nn.Conv1d`: 1D convolution with padding
- `nn.LSTM`: Bidirectional LSTM
- `nn.Sequential`: Chains attention layers
- `nn.Linear`: Fully connected layers
- `nn.Tanh`: Activation function
- `nn.Dropout`: Regularization
- `nn.init.xavier_uniform_`: Weight initialization

Functional Interface (F)

- `F.relu`: Activation function
- `F.max_pool1d`: Max pooling
- `F.softmax`: Attention weights normalization

Hybrid Architecture Benefits

- **CNN path**: Captures local patterns and n-grams (fast, parallel)
- **BiLSTM path**: Captures long-range dependencies and context (sequential)
- **Attention**: Focuses on important time steps in LSTM output
- **Feature fusion**: Combines strengths of both architectures